

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	3 rd CS & BS	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

95

Code of Conduct

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	✓			
Constraint Analysis	25%	✓			
Solution Development	25%	✓			
Application of Concepts	15%		✓		
Justification	10%				
Total Marks (100):		95			

Faculty In-charge	Course Coordinator	Head of Department
Name: <u>P. S. Suleen</u>	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: <u>21/10/24</u>	Date: _____	Date: _____

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.

(BL5,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks: 50

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Constraint-Based Problem Solving

Q90. Constraint-Based Course Prerequisite Scheduling

Problem

The objective is to schedule a set of courses across a fixed number of semesters while satisfying all prerequisite relationships.

For example, if **Course A** $\rightarrow$ **Course B**, then A must be completed before B.

Constraints

1. A course can be taken only after all its prerequisites are completed.
2. Each semester has a maximum course limit.
3. A course cannot be repeated.
4. All required courses should be scheduled.
5. The number of semesters is limited.

Backtracking Approach

The algorithm assigns courses semester by semester.

Schedule(semester):

 if all courses are scheduled:

 print schedule

return true

 for each unscheduled course:

 if all prerequisites are completed:

 if semester has available capacity:

 add course to current semester

 if Schedule(semester + next position):

 return true

 remove course // Backtrack

return false

Pruning

Invalid choices are removed early:

- Do not select a course whose prerequisite is incomplete.
- Do not exceed the semester course limit.
- Stop if remaining semesters cannot accommodate remaining courses.
- Detect dependency cycles before scheduling.

Correctness

The algorithm places a course only when its prerequisites are satisfied. Since every possible valid course choice is explored and invalid choices are discarded, any schedule produced satisfies all constraints.

Complexity

Course prerequisite scheduling can be formulated as a difficult combinatorial problem when additional constraints such as semester limits and course conflicts are included. In the general constrained version, the search space can be **exponential**, approximately **$O(n!)$** without strong pruning.

Conclusion

Backtracking provides a systematic way to generate valid schedules. Constraint checking and pruning significantly reduce unnecessary exploration.

Q91. Maximum Coverage Problem

Problem

Given a collection of subsets and a fixed value **k**, select at most/exactly k subsets so that the number of unique covered elements is maximized.

Example:

$$S1 = \{1,2,3\}$$

$$S2 = \{2,4,5\}$$

$$S3 = \{5,6\}$$

$$k = 2$$

Choosing S1 and S2 covers $\{1,2,3,4,5\} \rightarrow 5$ elements.

Constraints

1. Exactly or at most k subsets can be selected.
2. A subset can be selected only once.
3. Only unique elements are counted.
4. The objective is to maximize coverage.

Backtracking Approach

MaxCoverage(index, selected, covered):

if selected == k:

 update maximum coverage

 return

if index == number_of_sets:

 return

// Select current subset

newCovered = covered $\cup$ Set[index]

 MaxCoverage(index + 1, selected + 1, newCovered)

// Do not select current subset

 MaxCoverage(index + 1, selected, covered)

Pruning

A branch can be stopped when:

- k subsets have already been selected.
- There are not enough remaining subsets to reach k.
- The theoretical maximum possible coverage from remaining subsets cannot improve the current best.
- A selected combination is clearly dominated by another combination with equal or greater coverage.

Correctness

The algorithm considers both possibilities—selecting and not selecting every subset. Therefore, every feasible combination of k subsets is considered. The combination with maximum unique coverage is stored as the optimal solution.

Complexity

For **n subsets**, there are approximately:

2^n possible

combinations.

Therefore, the worst-case time complexity is: $O(2^n \times m)$ where m represents the cost of calculating coverage.

The optimization version is **NP-Hard**.

Conclusion

Backtracking guarantees the optimal answer but can become expensive for large numbers of subsets. Pruning improves practical performance.

Q92. Constraint-Based Puzzle Arrangement Problem

Problem

The objective is to arrange puzzle pieces in a valid configuration so that adjacent edges match according to predefined rules.

For example:

Piece A right edge = Piece B left edge must be satisfied if B is placed immediately to the right of A.

Constraints

1. Each piece can be used only once.
2. Adjacent edges must match.
3. Pieces must occupy valid positions.
4. Orientation/rotation rules must be satisfied.
5. All required positions must be filled. **Backtracking Approach**

Arrange(position):

if all positions are filled:

return true

for each unused piece:

for each valid orientation: if

piece matches neighboring edges:

place piece

if Arrange(next position):

return true

```
remove piece    // Backtrack
```

```
return false
```

Pruning

Before placing a piece:

- Check its left edge against the previous piece.
- Check its upper edge against the piece above it.
- Reject invalid rotations.
- Reject already-used pieces.
- Stop immediately when an edge mismatch occurs.

Correctness

A piece is placed only when all currently required constraints are satisfied. The algorithm explores every possible piece and orientation when necessary. Thus, if a valid arrangement exists, backtracking can find it.

Complexity

For **n** pieces, the number of arrangements can approach:

O(n!) and rotations can increase the search space further.

The general constraint arrangement problem is **NP-Complete** when expressed as a decision problem.

Conclusion

Backtracking is suitable because incorrect arrangements can be rejected immediately. Edge matching provides strong pruning.

Q93. Minimum Cost Flow with Constraints

Problem

The goal is to send required flow through a network while minimizing the total transportation cost.

Each edge has:

- Capacity
- Cost
- Flow

Constraints

1. Flow must not exceed edge capacity.
2. Flow must be non-negative.

3. Flow conservation must hold at intermediate nodes.
4. Supply and demand requirements must be satisfied.
5. Total cost should be minimized. **Backtracking Approach**

AssignFlow(edge):

```

    if all edges are assigned:      if all demands and
conservation constraints are satisfied:
        update minimum cost
return

```

```

for flow = 0 to capacity(edge):

```

```

    if flow is feasible:
        assign flow
        AssignFlow(next edge)
remove flow    // Backtrack

```

Pruning

A branch is rejected if:

- Edge capacity is exceeded.
- Node flow conservation becomes impossible.
- Demand cannot be satisfied with remaining capacity.
- Current cost already exceeds the best known cost.
- Remaining network cannot provide required flow.

Correctness

The algorithm explores possible flow assignments and accepts only assignments satisfying capacity, conservation, supply, and demand constraints. Among valid assignments, the minimum-cost solution is selected.

Complexity

If an edge has capacity C and there are E edges, naive enumeration may require roughly: $O((C+1)^E)$ possibilities.

With integer capacities, the constrained variant can have exponential search complexity. However, it is important to note that **ordinary minimum-cost flow is polynomial-time solvable**; the NP-Hard classification applies to certain additional constrained variants, not the standard problem.

Conclusion

Backtracking can solve small constrained flow problems, while specialized flow algorithms are preferable for large networks.

Q94. Constraint-Based Robot Motion Planning

Problem

A robot must move from a starting position to a goal while avoiding obstacles and satisfying movement constraints.

Example:

Start → Free cell → Free cell → Goal

↓

Obstacle X

Constraints

1. Robot cannot enter obstacles.
2. Robot must remain within the environment.
3. Each movement must be valid.
4. The path must be continuous.
5. The robot should reach the goal.
6. Additional constraints may restrict direction, turning angle, energy, or distance.

Backtracking Approach

FindPath(position):

if position == goal:

 return true

mark position as visited

for each possible movement:

next = position + movement

if next is valid and not visited:

 add next to path

```
if FindPath(next):
```

```
    return true
```

```
    remove next from path
```

```
return false
```

Pruning

Reject a movement when:

- It enters an obstacle.
- It goes outside the environment.
- The position has already been visited.
- It violates movement constraints.
- The remaining distance/cost cannot produce a better solution.

Correctness

The algorithm explores every feasible movement from the current position. Unsafe movements are rejected. Therefore, if a valid path exists within the explored state space, the algorithm can find it.

Complexity

For a graph with **V vertices** and **E edges**, basic path search can be handled efficiently. However, realistic motion planning becomes much harder when continuous positions, multiple obstacles, robot configurations, and additional constraints are included.

The generalized constrained motion-planning problem is **NP-Hard**.

Conclusion

Backtracking is useful for small environments and discrete motion planning. For large environments, techniques such as A*, Dijkstra's algorithm, RRT, or other motion-planning methods are more scalable.

Q95. Maximum k-Colorable Subgraph Problem

Problem

Given a graph $G(V,E)$ and k colors, find the largest subset of vertices whose induced subgraph can be properly colored using at most **k colors**.

A proper coloring means adjacent vertices cannot have the same color.

Constraints

1. Every selected vertex must receive a color.
2. Only k colors are available.

3. Adjacent selected vertices must have different colors.
4. Unselected vertices do not need coloring.
5. The number of selected vertices should be maximized. **Backtracking Approach**

ColorSubgraph(index):

```

if index == number_of_vertices:
    update maximum selected vertices
    return

// Try selecting the vertex for color =
1 to k:    if color is valid for selected
neighbors:
    select vertex
    assign color

    ColorSubgraph(index + 1)

    remove vertex
remove color

// Try excluding the vertex
ColorSubgraph(index + 1)

```

Pruning

A branch is rejected when:

- A selected vertex conflicts with a neighbor having the same color.
- The remaining vertices cannot improve the current best solution.
- A color assignment violates the k-color limit.
- An upper bound on possible additional selected vertices is no better than the current solution.

Correctness

The algorithm considers both choices for every vertex: **include or exclude**. For included vertices, all possible colors are tested. Invalid color conflicts are rejected. Therefore, every feasible k-colorable subgraph can be considered, and the largest one is retained.

Complexity

For each of **n vertices**, there can be up to **k color choices** plus the possibility of excluding the vertex.

Thus, the worst-case search space can be approximately:

$$O((k+1)^n)$$

Hence the problem has **exponential complexity** in the general case and is **NP-Hard**.

Conclusion

Backtracking guarantees an optimal solution for small graphs. Pruning based on color conflicts and upper bounds can greatly reduce the practical search space.